Universitat de Lleida

TREBALL DE FI DE MÀSTER

Estudiant: **RAUL HIDALGO CABALLERO**

Titulació: **Màster en Enginyeria Informàtica**

Títol de Treball de Fi de Màster: **WEBRTC IN RADIOCONTROLLED CAR VIA 5G**

Director/a: **David Sarrat González**

Presentació

Mes: Setembre

Any: 2026

Acknowledgements:

Quisiera agradecer a todos aquellos que me han visto estar a punto de rendirme con este proyecto y que, aun así, nunca han dejado de creer en mí, incluso cuando ni yo mismo pensaba que conseguiría completar este prototipo.

A todos aquellos que me han ayudado con el diseño 3D, tanto del prototipo actual como del diseño pensado para el futuro.

A todos aquellos que esperan con ilusión la versión final y que, mes tras mes, me preguntan cómo avanza el prototipo.

Todos y cada uno de vosotros me habéis dado las fuerzas necesarias para seguir adelante cada vez que algo salía mal, cada vez que fracasaba y cada vez que pensaba en rendirme.

Gracias por creer en este proyecto y, sobre todo, por seguir creyendo en mí cuando a mí me costaba hacerlo.

Abstract:

This work describes the design, development, and evaluation of a remotely operated radio-controlled (RC) vehicle connected to the Internet via a 5G mobile network. The system enables real-time video transmission and vehicle control through a web-based interface. An onboard camera captures live video, which is streamed to a remote client using Web Real-Time Communication (WebRTC), selected for its low-latency, peer-to-peer communication capabilities.

The system architecture is composed of a backend implemented with ASP.NET Core, responsible for signaling, session management, and secure communication, and a frontend developed using React, which provides a browser-based user interface for video visualization. The use of 5G connectivity provides high bandwidth and low latency, which are essential for maintaining stable video streaming and responsive control in real-time teleoperation scenarios.

An experimental evaluation is conducted to assess system performance, focusing on end-to-end latency and video transmission stability. The results demonstrate that the proposed solution is technically viable and that the integration of WebRTC over 5G networks is suitable for remote control applications. This work confirms the potential of the proposed architecture as a foundation for future developments in teleoperation, connected robotics, and Internet of Things (IoT) systems.

Table of Contents:

Table of Figures:

Introduction:

Remote operation of a physical vehicle through a web browser requires more than sending a live image over the Internet. The operator must see the consequences of movement soon enough to react, while steering and throttle data must travel in the opposite direction without being delayed behind obsolete commands. The perceived response time therefore depends on the complete chain: camera capture, encoding, mobile uplink, Internet routing, optional relaying, browser decoding, display, input processing, and actuation. A useful teleoperation platform must treat these stages as one end-to-end system.

Mobile connectivity makes this problem more difficult. A vehicle may operate behind carrier-grade Network Address Translation (NAT), encounter restrictive firewalls, and experience changing bandwidth or delay as radio conditions vary. The browser and the vehicle cannot assume that either endpoint has a stable, publicly reachable address. The system must therefore discover an available device, coordinate a session, exchange connection information, test candidate network paths, and use a relay when a direct path cannot be established. Fifth-generation mobile networks are the intended access context for this project, but their nominal capabilities do not determine the final user-visible delay; the full path must be measured on the working prototype.

Web Real-Time Communication (WebRTC) provides the real-time communication foundation used in this work. It combines media transport, congestion control, encrypted sessions, Interactive Connectivity Establishment (ICE), Session Traversal Utilities for NAT (STUN), Traversal Using Relays around NAT (TURN), and bidirectional data channels. WebRTC does not prescribe the application-specific mechanisms needed to publish device availability or exchange signaling messages. The proposed architecture therefore uses an ASP.NET Core server and SignalR for the coordination plane: device presence, operator selection, session state, Session Description Protocol (SDP) messages, and ICE candidates. After a route has been negotiated, video and data use the selected direct or TURN-relayed WebRTC path, keeping the application server outside the latency-sensitive media flow.

This Master's Final Project applies that architecture to an Internet-connected radio-controlled (RC) vehicle. The prototype combines a Tamiya TT-01 Type E chassis, a Raspberry Pi 5, a Logitech C920 camera, a mobile-network dongle, a .NET device process, an ASP.NET Core signaling service, and a React browser interface. On the vehicle, the current video pipeline captures MJPEG frames and encodes H.264 in software before delivering the stream through WebRTC. In the browser, the operator can select the prototype device, negotiate a session, view the live stream, and inspect the selected connectivity path.

Negotiated data channels establish the software structure for later control, command, telemetry, and event traffic.

The central engineering question is whether this browser-native architecture can deliver a live vehicle view with a glass-to-glass delay below 400 ms while preserving a practical path toward bidirectional control.

Objectives

The project is developed around the following objectives:

- To design a modular hardware and software platform that integrates an RC chassis, onboard computing, camera capture, mobile Internet access, signaling infrastructure, and a browser-based operator interface.
- To implement the coordination flow required to expose the prototype device, initiate an operator session, exchange SDP descriptions and ICE candidates, and recover cleanly from connection termination.
- To establish an encrypted WebRTC video path that can select a direct peer-to-peer route when available and use TURN relaying when network restrictions prevent direct connectivity.
- To build a low-latency camera-to-browser pipeline and assess its glass-to-glass delay against the project target of less than 400 ms.
- To define the data-channel structure for control, commands, telemetry, and events, and to identify the safety, actuation, networking, electrical, and mechanical work required to turn the proof of concept into a complete remotely driven vehicle.

Scope and Limitations

The implemented scope concentrates on the communication and video foundation. It covers the assembly of the prototype hardware, the Raspberry Pi device service, a prototype catalogue and operator-selection flow, SignalR-based signaling, WebRTC offer and answer negotiation, ICE candidate exchange, STUN and TURN configuration, live H.264 reception in a browser, connection-state observation, and resource cleanup. A browser-based test device is also used to exercise signaling, NAT traversal, and media presentation independently of the Raspberry Pi camera chain. The evaluation focuses on end-to-end architectural viability and on a glass-to-glass latency observation from the assembled prototype.

Several limits are material to the interpretation of the result. The measured value comes from one hardware configuration and a representative test setup; it is not a statistical characterization across mobility, coverage levels, packet loss, jitter, or every direct and relayed route. Although 5G is the target

network for the project, the present iteration uses a Huawei 4G dongle for cost reasons. The 340 ms observation must therefore be attributed to the tested end-to-end configuration rather than to 5G technology. Hardware H.264 capture, systematic IPv4 and IPv6 comparisons, and testing on 5G Non-Standalone and Standalone networks are reserved for later evaluation.

The current software detects a gamepad and creates negotiated WebRTC data channels, but it does not yet implement the complete browser control loop, device-side validation of control messages, or physical steering and propulsion. Fail-safe behavior for stale commands, link loss, invalid input, and emergency stopping has consequently not been validated on the vehicle. Application-level authentication and production-ready electrical and mechanical integration also remain outside this proof of concept. These boundaries mean that the work validates remote observation and the underlying communication design, not finished remote driving.

Document Structure

The remainder of this document develops the project from the physical platform to the measured result. The Hardware chapter describes the chassis, Raspberry Pi, camera, mobile connection, power supply, and component-selection criteria. General Architecture explains the device, application server, TURN server, browser, and the complete signaling and media flow. The WebRTC chapter presents the connection model, transport stack, data channels, codecs, and related interoperability mechanisms. Client documents the Raspberry Pi process, video pipeline, session management, and channel design, while Server covers the ASP.NET Core and React components used for discovery, signaling, browser playback, and validation. The final chapters interpret the latency result, state the conclusions, and define the work required to complete and evaluate the remote-control system.

Motivation:

I chose this project because I wanted to recover the enjoyment that first drew me to programming. I still like writing software, but my work had increasingly become a routine of emails and coordination, leaving less room for the practical work I enjoyed. I wanted a project of my own that would give me a reason to experiment again. Building something physical offered a way to step away from that routine and see software produce a result I could observe directly.

An RC vehicle gave that desire a concrete form. A camera mounted on a real chassis turns a communication problem into something visible: an image has to travel from the vehicle to the operator, and any delay becomes part of the experience. Working towards remote operation also brings software into contact with power supplies, mechanical assembly, and the limitations of a mobile connection. This combination appealed to me because progress would depend on making the different parts work together in a tangible prototype.

There is also a more personal reason behind the choice. When I was a child, I wanted to become a combat pilot. That is a dream I do not expect to fulfil, but the interest in flying and operating a vehicle from its own point of view has stayed with me. Creating a system that lets me see through an onboard camera gives that interest a practical direction. It is something I can build with the programming knowledge I have and the technical skills I can develop through the project.

The RC car gives this ambition a manageable starting point. The prototype allows me to learn about the communication path on the ground, while leaving vehicle control and more demanding applications for later stages. Flight remains a personal aspiration beyond the scope of this thesis. For now, the achievement I seek is a working foundation: a physical vehicle whose live view can reach a remote browser with a measured and useful response time. I need to learn to walk before I can fly, and this project is my first step.

Initial Design

The project began with browser experiments before the camera pipeline was moved to the physical vehicle. This starting point provided a small environment in which to explore the WebRTC connection model and distinguish media transport from the messages needed to establish a session. The earliest recorded test, in February 2026, created two RTCPeerConnection objects in the same browser tab. Camera and microphone tracks were captured locally, while session descriptions and ICE candidates were exchanged directly between the two objects. The received stream was displayed beside the local preview.

The next experiments separated the caller and callee, followed by distinct video sender and receiver pages. These pages used BroadcastChannel to exchange signaling messages between browser contexts and configured a public Google STUN server. They remained local browser experiments: BroadcastChannel did not provide the Internet signaling service required for an independently connected vehicle. Nevertheless, the separation made the publishing and viewing responsibilities explicit before introducing a central application server.

The first networked architecture combined a React browser frontend with an ASP.NET Core backend and a SignalR hub. SignalR replaced the local exchange mechanism with a persistent connection through which peers could discover one another and forward session descriptions and ICE candidates. A browser served as the video source during this stage, while another browser received and displayed the stream. This provided a prototype of the eventual device-to-browser interaction without yet requiring native camera capture on the Raspberry Pi.

The component responsibilities were distinct. The publishing endpoint acquired media and added its tracks to a WebRTC peer connection. The receiving endpoint negotiated compatible media parameters and attached the remote stream to a video element. The application server carried the coordination messages that allowed these endpoints to establish their connection. It did not decode, encode, or redistribute the video. This distinction between signaling and media remained central to the architecture as the prototype evolved.

STUN formed part of the initial connectivity configuration. It allowed ICE to discover server-reflexive candidates in addition to local addresses, providing possible paths for communication across address translation. However, discovering a public-facing address did not establish that the other endpoint could reach it. The initial design therefore represented a direct-connection

starting point rather than a guarantee of connectivity across every mobile carrier, router, or firewall. TURN relay configuration was added later as the implementation developed.

The physical vehicle introduced a further integration stage. The intended onboard role was to capture the camera image and publish it to the operator through the same general WebRTC model. The native .NET device service and its Raspberry Pi camera integration appeared later in the repository history. They should therefore be understood as developments of the browser prototype, rather than components already present in the earliest experiments.

This progression established the design's main boundary: a central service for signaling, with a separate media path between publishing and viewing endpoints, directly or through a relay when required. The Hardware chapter describes the selected physical components, while General Architecture explains the resulting system and its connection flow. The initial design presented here provides the starting point for that final architecture and for the implementation steps described later.

Hardware:

The hardware architecture is composed of an RC vehicle chassis, a Raspberry Pi, a camera module, and a 5G communication device. Together, these components enable the vehicle to stream live video to a remote client and receive driving commands from the web interface.

The system is divided into several hardware subsystems, each responsible for a specific function. This modular approach makes the platform easier to assemble, test, maintain, and improve during later development stages.

Hardware Architecture:

RC Vehicle Chassis

The base of the project is a radio-controlled car chassis that provides the mechanical structure, wheels, suspension, motor, and steering system, batteries. This chassis allows the project to focus on connectivity, control, and video transmission rather than on the design of a vehicle from scratch.

The exact chassis that we will use as a base is a "Tamiya TT01 Type E", one of the most common chassis available.

In the following git repository, there are most of the design files that form the chassis with assembly instructions ready to print with a 3D printer: https://github.com/scarlet-radio-control/tt-01-type-e

Figure 1 Tamiya Chassis

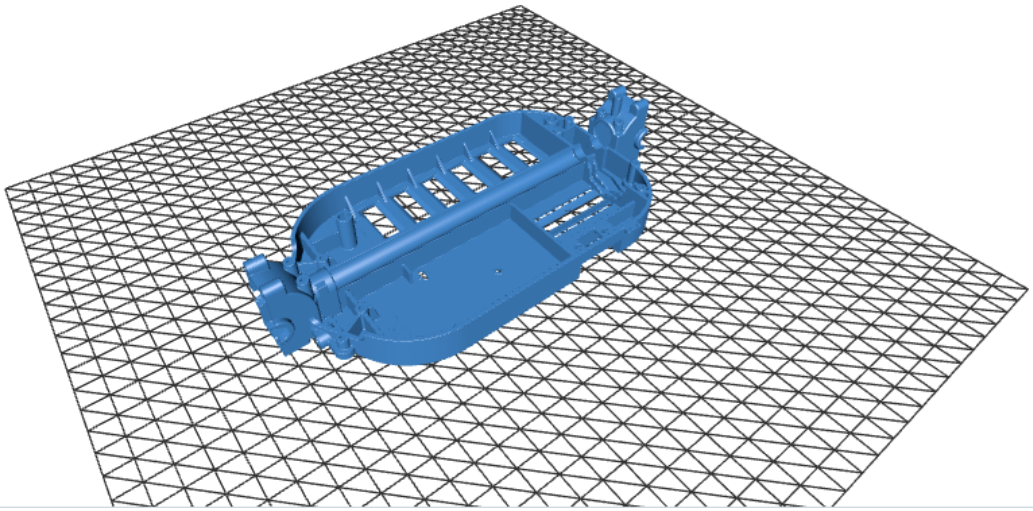

Figure 2 STL Chassis

Raspberry Pi 5:

A Raspberry Pi 5 is installed on the vehicle to coordinate the local execution of the system. Its main responsibilities are to capture the video stream, manage communication with the remote application, and process the control commands received from the client. This unit acts as the bridge between the software and the chassis.

The Raspberry Pi 5 was selected due the fact that high quality WebUSB cameras were a requirement, and other alternatives like an ESP32 were unavailable due to throughput limitations.

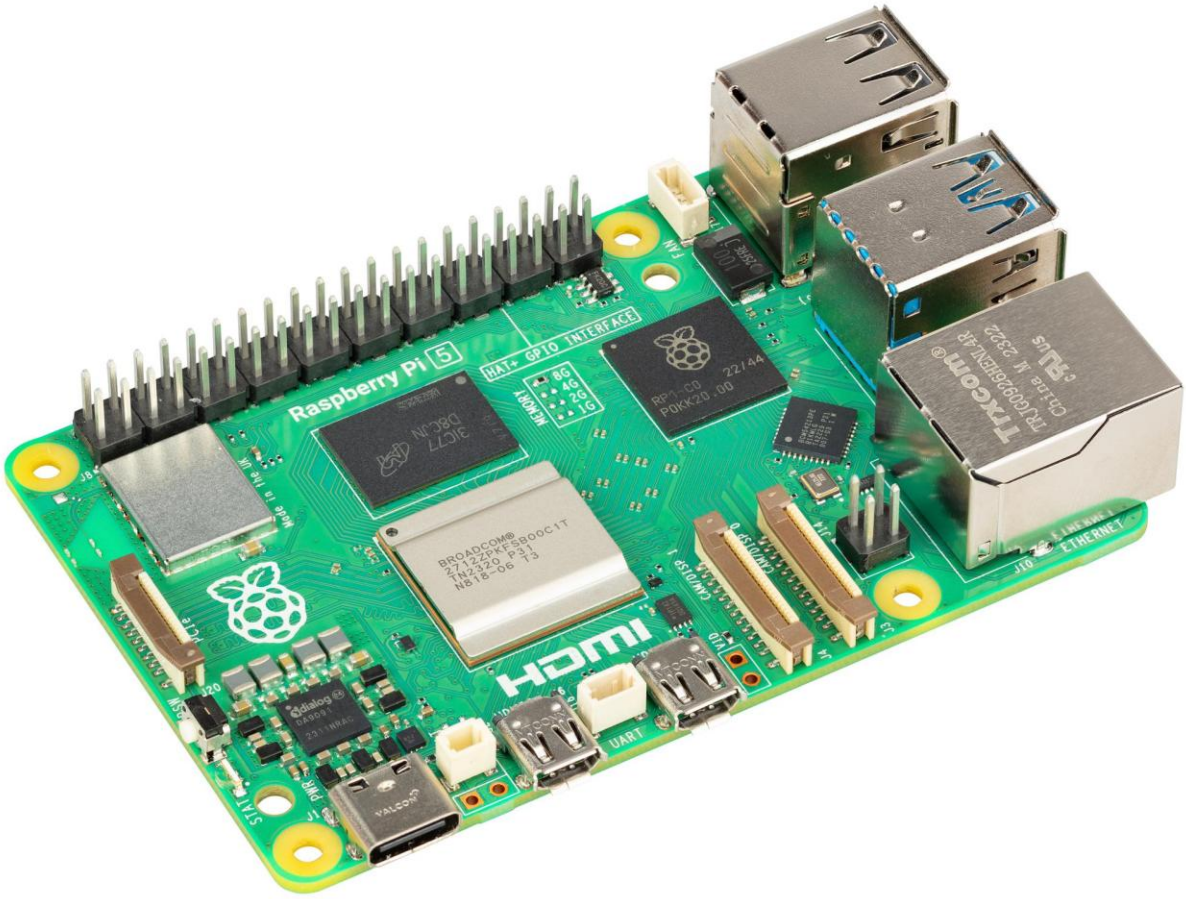

Figure 3 Raspberry Pi

Camera:

The camera is responsible for capturing the live video feed from the vehicle's perspective. It is mounted on the chassis in a forward-facing position to provide the remote operator with a clear view of the driving environment. The placement of the camera is important because it directly affects usability, depth perception, and the operator's ability to react to obstacles.

A Logitech HD Pro C920 was chosen because it had built in hardware encoding of the codecs that will be required for WebRTC, a USB interface, support for Linux Video stack, and good image quality.

Figure 4 Logitech HD Pro C920

5G Connectivity Dongle:

The vehicle uses a 5G mobile connection to access the Internet outside a local Wi-Fi environment. This connection is essential for remote operation, as it provides the bandwidth required for video streaming and the low latency required for responsive control. The 5G device can be integrated as a modem, mobile router, or tethered connection, depending on the final implementation.

For this iteration a Huawei 4G dongle was used. From the point of view of the system, it's just an internet connection. This is solely a price decision

Figure 5 Huawei 4G Dongle

Power Supply System:

For the power supply I will use the standard NiMH battery common on RC cars.

Also to adapt the voltage to the Raspberry Pi a small DC-DC voltage regulator is also attached and serves like the electrical interface between the battery, the Raspberry Pi, and the chassis.

Figure 6 NiMH Battery

Design Considerations:

The hardware components were selected with the objective of building a functional proof of concept rather than a fully optimized commercial vehicle. For this reason, priority was given to components that were easy to integrate, configure, replace, and test. The main goal was to validate the complete system architecture: remote video transmission, 5G connectivity, browser-based interaction, and real-time vehicle control.

Each hardware element was chosen to reduce integration complexity. The use of an existing RC vehicle chassis avoids the need to design a mechanical platform from the beginning and provides a ready-to-use base with motor, steering, wheels, and suspension. This allows the development effort to focus on the communication and control layers of the project.

The Raspberry Pi was selected to act as a flexible bridge between the software system and the physical vehicle. Its role is not only to process the video stream and control commands, but also to simplify the connection between the different subsystems. By using a general-purpose computing platform, the system can be modified and tested more easily during development.

The camera was also selected with integration in mind. A simple camera interface reduces configuration effort and allows the video pipeline to be tested quickly. Since the project depends on real-time visual feedback, the camera does not need to represent the most advanced imaging solution available, but it must be reliable enough to validate the WebRTC streaming architecture.

The 5G connectivity device was included to demonstrate the feasibility of operating the vehicle outside a local network. In the context of this proof of concept, the important factor is not the specific modem or router model, but the ability to provide mobile Internet access with sufficient bandwidth and latency for video streaming and remote control.

Power distribution was considered from the perspective of stability and practicality. The additional electronic components must be powered without interfering with the original RC vehicle electronics. Therefore, the power system should be simple, accessible, and reliable enough to support repeated testing sessions.

Overall, the hardware design follows a modular philosophy. Each component has a clear responsibility and can be tested independently before being integrated into the complete system. This approach makes the prototype easier to assemble and debug, while also allowing future improvements such as a better camera, additional sensors, a more compact processing unit, or a more advanced control board.

As a result, the selected hardware provides a practical platform for validating the main idea of the project: that an RC vehicle can be remotely operated through a web interface using WebRTC video transmission and 5G connectivity. The emphasis is therefore placed on demonstrating the viability of the complete end-to-end system rather than on optimizing each individual hardware component.

General Architecture:

The complete system follows a distributed architecture composed of four main elements: the device, the application server, the TURN server, and the player. The device is the RC car and contains the onboard computing, video, communication, and actuation interfaces. The browser is the client-side application executed in the remote operator's web browser. The application server and TURN server provide the supporting infrastructure required to coordinate the session and establish the WebRTC connection across different networks.

The design separates coordination traffic from latency-sensitive real-time traffic. The application server handles device registration, session management, and WebRTC signaling. After negotiation, video and control data are exchanged between the RC car and the player through the route selected by Interactive Connectivity Establishment (ICE). A direct peer-to-peer route is preferred; the TURN server is used as a relay when the mobile network, carrier-grade NAT, or a firewall prevents direct connectivity. As a result, the application server is not required to process the media stream.

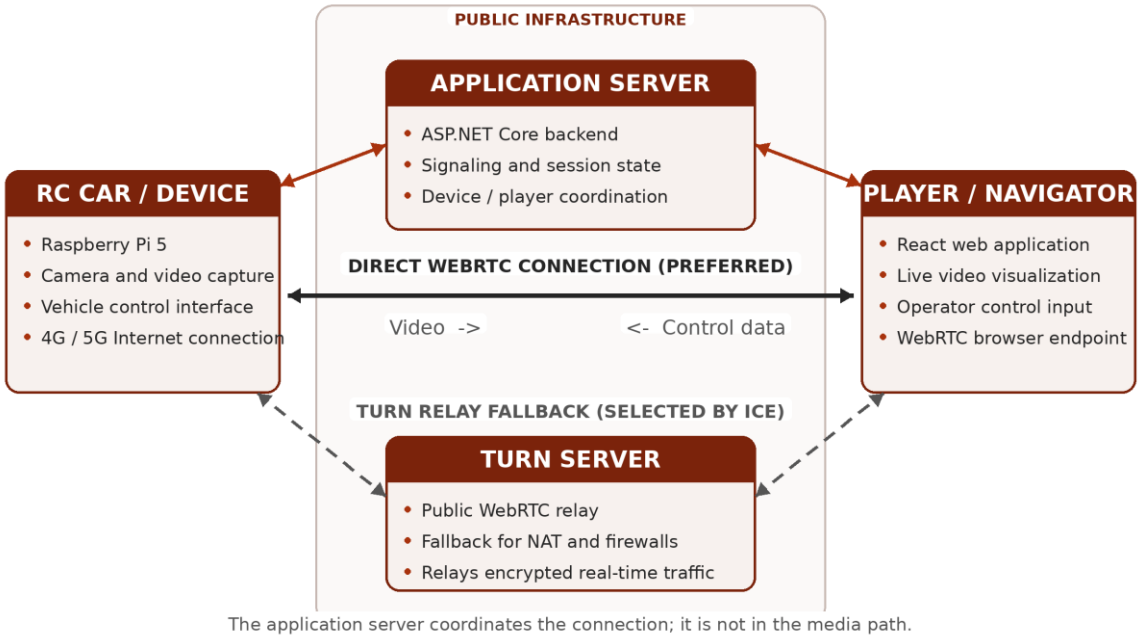

Figure 7 Signaling

Architecture Components:

Device (RC Car):

The device is the physical endpoint installed on the vehicle. The Raspberry Pi 5 runs the local application, captures the stream from the forward-facing camera, creates the WebRTC peer connection, and interfaces with the motor

and steering electronics. It reaches the Internet through the mobile connection and maintains a signaling connection with the application server so that it can announce its availability and receive session requests. During an active session, the device acts as the video producer and the consumer of the driving commands.

Application Server:

Implemented with ASP.NET Core, the application server provides the coordination layer. It keeps track of connected devices and players, creates and closes sessions, and forwards the WebRTC negotiation information exchanged by both endpoints, including session descriptions and ICE candidates. It also provides the point at which authentication and access rules can be applied. Once the peer connection is established, the server remains responsible for session state but is normally outside the video and control-data path.

TURN Server:

The TURN server is a separate, publicly reachable service whose purpose is to relay WebRTC traffic when a direct route is unavailable. This fallback is important for vehicles because mobile operators frequently place clients behind carrier-grade NAT. The endpoints first perform connectivity checks; if no direct candidate succeeds, ICE selects a relay candidate supplied by the TURN server. In that case, the encrypted video and control data pass through the relay, while the application server continues to handle signaling and session management.

Browser:

The player is the React application that opened in the remote operator's web browser. It displays the live camera stream, presents the state of the connection, and captures the operator's steering and throttle input. The browser creates the second WebRTC peer connection and sends the driving commands to the vehicle through a WebRTC data channel, while receiving the video track in the opposite direction. Because the player runs in a standard browser, the system does not require a dedicated desktop application.

End-to-End Communication Flow:

Device availability and discovery. When the onboard application starts, the RC car establishes Internet connectivity through the 5G modem and opens its signaling connection to the application server. It then registers itself as an available device. The server maintains the association between the device identifier and the active signaling connection, allowing the web application to discover that the car is online. At this stage, no video or control traffic is exchanged; the server is only maintaining presence and session state.

Session initiation. The operator opens the React application in the browser and requests a session with the selected vehicle. The application server validates the request according to the configured access rules, associates the player with the device, and notifies both endpoints that negotiation can begin. Keeping this step on the server prevents the two endpoints from needing to know each other's public network address in advance and provides a central place for authentication, authorization, and session lifecycle management.

WebRTC signaling. One endpoint creates an SDP offer describing the proposed WebRTC session, including the video track, supported codecs, transport parameters, and data channel. The application server forwards the offer to the other endpoint, which creates and returns an SDP answer. The server acts only as a trusted message broker during this exchange: it transports the descriptions required to configure the peer connections but does not receive or process the camera frames or driving commands.

ICE candidate exchange and path selection. In parallel with the SDP exchange, the device and player gather and exchange ICE candidates through the signaling channel. These candidates describe the network routes that may be used, such as local interfaces, publicly reachable addresses discovered through NAT traversal, and relay addresses allocated by the TURN server. ICE connectivity checks test the candidate pairs and select the highest-priority working route. A direct device-to-player path is preferred because it avoids an additional relay hop. If carrier-grade NAT on the mobile network, a firewall, or another restrictive network policy prevents direct connectivity, the endpoints use the TURN relay instead.

Secure transport establishment. After ICE has selected a viable route, WebRTC establishes its encrypted transports. The video stream is protected with SRTP using keys negotiated by DTLS, while control messages are carried by an SCTP data channel protected by DTLS. Consequently, the security properties of the real-time connection do not depend on whether traffic follows a direct route or passes through the TURN server; the relay forwards encrypted packets and does not terminate the WebRTC session.

Bidirectional real-time operation. During the active session, the Raspberry Pi captures and encodes frames from the onboard camera and sends the resulting video track to the player. The browser receives, decodes, and renders the stream for the operator. In the opposite direction, steering and throttle input collected by the user interface is serialized into compact control messages and transmitted over the WebRTC data channel. The device receives these messages and converts them into commands for the steering and motor interfaces. Media and control therefore share the negotiated WebRTC route but remain logically separate streams with different timing and reliability requirements.

Monitoring, recovery, and termination. The signaling connection remains active throughout the session so that the server can track participant state and communicate errors, renegotiation requests, or disconnection events. WebRTC monitors the selected transport and exposes connection statistics that can be used to observe latency, packet loss, jitter, and available bitrate. Temporary network variation is handled by the WebRTC congestion-control and buffering mechanisms; a longer route interruption may require a new ICE negotiation or termination of the session. When either participant disconnects, the server closes the logical session, the peer connections and media resources are released, and the device returns to the available state if its signaling connection remains active.

End-to-end paths and latency. The video path begins at camera capture on the vehicle and continues through encoding, 5G uplink transmission, the selected direct or relayed Internet route, browser decoding, and final display. The control path begins with operator input in the browser and continues through event processing, message serialization, WebRTC transmission, device-side parsing, and actuation. The application server is outside both latency-sensitive paths after negotiation, although the TURN server adds a network hop when relaying is required. This distinction is important for evaluation because measured end-to-end delay includes processing and presentation time as well as network propagation time.

Communication planes. The complete flow therefore consists of two logical planes. The coordination plane uses the application server for registration, discovery, signaling, and session management. The real-time plane uses the WebRTC connection for vehicle-to-player video and player-to-vehicle control, with the TURN server participating only when a direct route cannot be established. Separating these responsibilities reduces server bandwidth requirements and makes the prototype easier to test, maintain, scale, and extend.

WebRTC:

Web Real-Time Communication (WebRTC) is not a single wire protocol. It is an ecosystem of browser APIs, media formats, transport protocols, security mechanisms, and connectivity procedures that cooperate to establish low-latency sessions between endpoints. The W3C defines the browser-facing objects, including RTCPeerConnection, media tracks, RTP senders and receivers, and RTCDataChannel, while the IETF specifies the protocols used on the network. This separation lets an application decide how users discover one another and how session descriptions are exchanged while retaining an interoperable real-time transport once the peers are connected.

A WebRTC session normally contains three logical activities. Signaling coordinates the participants and exchanges configuration data. Connectivity establishment discovers and tests possible network paths. The real-time plane then carries encrypted media and application data over the selected path. Signaling can use HTTPS, WebSocket, SIP, XMPP, or another application-defined mechanism; it is intentionally outside the WebRTC transport specification. Consequently, a signaling server may introduce the peers without becoming part of the latency-sensitive media path.

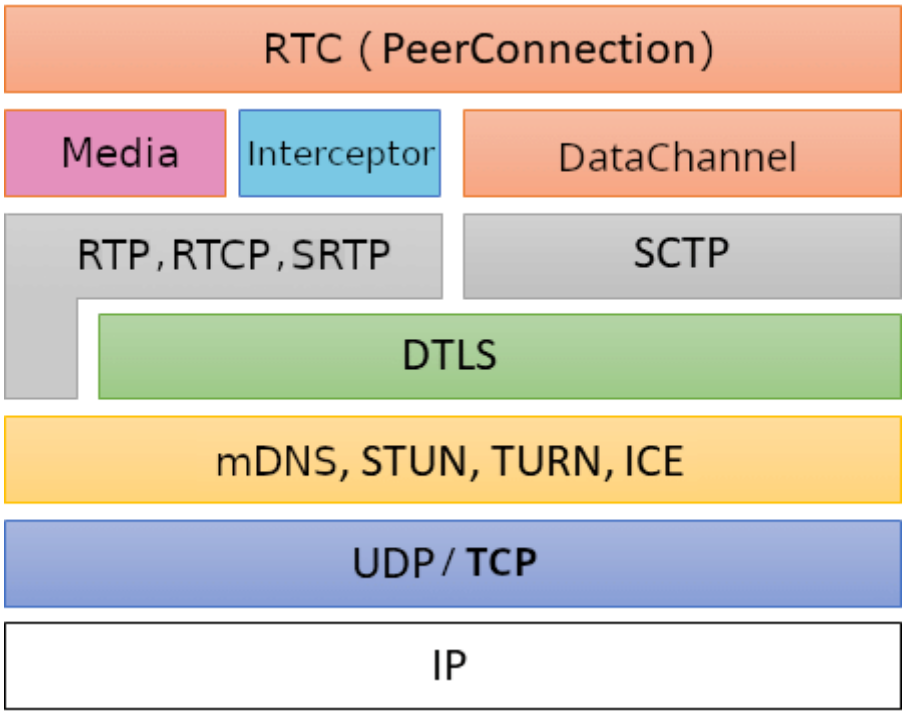

Figure 8 WebRtc protocol Stack

WebRTC Model and Connection Establishment

Signaling and SDP. Session negotiation begins with the offer/answer model. One endpoint creates an SDP offer that describes the media sections it wants to establish, the codecs and RTP parameters it supports, transport credentials, DTLS fingerprints, and whether data channels are required. The remote endpoint applies that offer and returns an SDP answer containing the compatible subset. JavaScript Session Establishment Protocol (JSEP) defines how browser applications drive this state machine through RTCPeerConnection without requiring the application to parse or rewrite SDP directly. Renegotiation can later add or remove tracks, change direction, or react to a network transition.

ICE and candidate selection. Interactive Connectivity Establishment (ICE) handles the fact that endpoints are commonly hidden behind Network Address Translators (NATs), mobile carrier-grade NAT, or firewalls. Each endpoint gathers candidate addresses and exchanges them through the signaling channel. ICE forms candidate pairs, prioritizes them, and performs authenticated STUN connectivity checks in both directions. A successful pair is nominated and becomes the path used by the WebRTC transports. Gathering and sending candidates as soon as they are discovered is known as trickle ICE; it allows path testing to overlap SDP exchange and generally reduces setup time.

Candidate types. A host candidate represents an address available directly on a local interface. A server-reflexive candidate represents the public address and port observed by a STUN server after NAT translation. A peer-reflexive candidate is learned dynamically during connectivity checks when a peer observes a mapped address that was not previously signaled. A relay candidate is an address allocated on a TURN server. Modern browsers may conceal some host addresses with multicast DNS names, but the candidate types and the ICE decision process remain the same. Candidate priority normally favors a working direct route over a relay because the direct route avoids an additional network hop.

STUN and TURN. Session Traversal Utilities for NAT (STUN) is a request/response protocol used by an endpoint to learn how a NAT maps its local socket and by ICE to test candidate-pair reachability. STUN does not forward the media stream and is therefore not a bandwidth relay. Traversal Using Relays around NAT (TURN) extends the same protocol family by allocating a public relay address and forwarding packets between a client and its peer. TURN is required when direct checks fail because of symmetric NAT behavior, restrictive enterprise policy, or blocked UDP. Relaying improves connection

success but consumes server bandwidth and can increase latency, so production systems normally treat it as a controlled fallback rather than the first choice.

Connection topologies. The selected topology depends on the application. A direct peer-to-peer connection is efficient for one-to-one communication. A Selective Forwarding Unit (SFU) receives encrypted media streams and forwards selected layers or copies to other participants without composing them, which scales group conferences more effectively. A Multipoint Control Unit (MCU) decodes, mixes, and re-encodes media to create a composite stream at the cost of processing and added delay. Streaming services often use media servers at ingress or egress, while still using the same ICE, DTLS, RTP, and congestion-control foundations.

Transport Stack and Data Channels

Once ICE selects a candidate pair, WebRTC multiplexes connectivity checks, secure media, and secure application data over the resulting transport. The two payload paths share connectivity and encryption infrastructure but use different protocols above it. The matrix below summarizes the principal layers.

WebRTC protocol stack at a glance

Layer	Media path	Data-channel path
Application	Audio/video tracks and RTP sender/receiver APIs	RTCDataChannel messages
Payload	VP8, H.264, VP9, AV1, or another negotiated codec	UTF-8 text or binary messages
Transport	RTP for media and RTCP for feedback and control	SCTP streams with ordered, unordered, reliable, or partial reliability
Security	SRTP/SRTCP; keys established by the DTLS-SRTP handshake	SCTP carried inside the authenticated DTLS connection
Connectivity	ICE-selected candidate pair; STUN checks; TURN relay when required	The same ICE-selected path and relay policy
Network	UDP/IP preferred; TCP or TURN transport fallbacks where supported	UDP/IP preferred; follows the negotiated ICE transport

Media transport and security. Media is packetized with the Real-time Transport Protocol (RTP). Its companion, RTCP, carries reception reports and feedback such as loss, jitter, round-trip timing, key-frame requests, and transport-wide congestion information. WebRTC uses the secure RTP profiles, so audio and video payloads and most control information are protected as SRTP and

SRTCP. Datagram TLS (DTLS) authenticates the endpoints using fingerprints exchanged in SDP and exports the keying material used by SRTP; the video frames themselves are not transported inside DTLS records.

Feedback and congestion control. Low latency requires the sender to adapt rather than allowing an unbounded delivery queue to form. RTCP feedback, packet-loss recovery, retransmission formats, key-frame requests, and bandwidth estimation let an implementation adjust bitrate, resolution, frame rate, or spatial and temporal layers. Buffers at the receiver smooth jitter, but excessive buffering increases glass-to-glass delay. These mechanisms are implementation-dependent, yet they operate within the common RTP and WebRTC transport requirements and are central to maintaining an interactive experience on variable networks.

Data channels. RTCDataChannel carries non-media application messages over Stream Control Transmission Protocol (SCTP), protected by DTLS. Multiple channels share a single SCTP association while retaining independent stream identifiers and delivery policies. The default channel is reliable and ordered, which is appropriate for chat, configuration, or commands that must arrive in sequence. Applications can request unordered delivery and can limit retransmissions or packet lifetime for time-sensitive state where a newer value makes an older one irrelevant. This partial-reliability model avoids head-of-line delay across obsolete messages while preserving message boundaries and congestion control.

Recovery and teardown. ICE consent checks continue after establishment so an endpoint does not send to a peer that no longer agrees to receive traffic. A network change can trigger candidate gathering or an ICE restart; a prolonged failure moves the connection to failed. Normal teardown closes the media senders, SCTP association, DTLS, and ICE transport, and releases TURN allocations and server-side resources.

Video Codecs

Codec negotiation. An endpoint advertises video codecs and format parameters in the SDP media section. Payload type numbers are session-local identifiers mapped through rtpmap, while fmtp carries codec-specific profiles and packetization settings. Offer and answer must select a common configuration before media flows. Applications can inspect implementation-dependent RTP sender and receiver capabilities and influence preference order, but cannot assume every browser, operating system, or hardware platform exposes the same optional encoders.

Practical comparison of WebRTC video codecs

Codec	WebRTC interoperability	Practical characteristics
VP8	Mandatory to implement	Broad interoperability and mature real-time software; hardware support varies by platform.
H.264/AVC	Constrained Baseline mandatory	Broad hardware acceleration; profiles and packetization must match, and licensing can matter.
VP9	Optional; implementation-dependent	Often better compression than VP8 and supports scalability; encode cost and hardware vary.
AV1	Optional; availability growing	High compression efficiency and scalability; demanding without hardware acceleration.
H.265/HEVC	Optional; fragmented support	Efficient and hardware-accelerated on some devices; interoperability and licensing require checks.

Interoperability baseline. RFC 7742 defines VP8 and H.264 Constrained Baseline as the mandatory-to-implement video codecs for WebRTC browsers and non-browser endpoints that support video. This baseline makes it likely that two compliant endpoints share at least one format, but it does not mean that every profile, level, resolution, or hardware path is compatible. VP9, AV1, and H.265 can improve compression or scalability in suitable deployments, yet they remain optional. Robust applications therefore negotiate from the intersection reported by both endpoints and avoid hard-coding a codec solely because it is available on the sender.

WHIP and WHEP

WHIP ingestion. WebRTC deliberately leaves application signaling open, which provides flexibility but complicates interoperability between independent streaming products. The WebRTC-HTTP Ingestion Protocol (WHIP) addresses the publisher-to-media-server direction. A WHIP client sends an SDP offer in an HTTP POST with content type application/sdp. The endpoint returns 201 Created with the SDP answer and a Location header identifying the session resource. HTTP PATCH can carry trickled ICE candidates or an ICE restart, and DELETE terminates the allocated session. WHIP was standardized as RFC 9725 in March 2025.

WHEP egress. The WebRTC-HTTP Egress Protocol (WHEP) applies a similar resource-oriented model to viewer playback. In the current draft, a player posts an SDP offer to the WHEP endpoint. The server may accept it with a 201 response containing an answer, or it may return a counter-offer that the player answers through the session resource. PATCH supports ICE-related updates and DELETE ends playback. WHEP defines a unidirectional media-server-to-viewer

profile rather than a general replacement for RTCPeerConnection signaling. As of August 2026, draft-ietf-wish-whep-04 is an active IETF Internet-Draft intended for the Standards Track; it must therefore be cited as work in progress, not as a published RFC.

Together, WHIP and WHEP constrain WebRTC's otherwise open signaling layer for interoperable one-way live streaming. Both establish a session resource through HTTP, use SDP for initial negotiation, PATCH for ICE-related updates, and DELETE for termination; their direction is the essential distinction: WHIP moves publisher media into a service, whereas WHEP delivers media from a service to a viewer. Once established, the underlying WebRTC session still uses ICE, DTLS, RTP/RTCP, SRTP, codec negotiation, and congestion control. They standardize session control without replacing WebRTC's media or transport stack.

Implementation:

Implementation progressed from browser-based WebRTC tests to an onboard video service. The February experiments first connected two peers within a single tab, then separated the sender and receiver into browser pages. This established the basic sequence of media capture, offer and answer negotiation, ICE candidate exchange, and playback. The ASP.NET Core SignalR hub was then introduced to carry those messages between independently connected endpoints, replacing the local BroadcastChannel mechanism used in the earlier tests.

As the network configuration developed, TURN services were added alongside STUN in March 2026. This provided relay candidates for ICE to test when direct connectivity was unavailable. The application server continued to handle signaling; a selected TURN relay belonged to the media transport path. Consequently, keeping video outside the ASP.NET Core service did not imply that every connection was direct. The browser interface also developed connection-state and candidate information to make the resulting session observable during testing.

The native device project was introduced in August, followed by camera integration using SIPSorcery for the WebRTC peer connection and SIPSorceryMedia.FFmpeg for capture and encoded video samples. The initial integration initialized FFmpeg within the .NET process and used FFmpegCameraSource to obtain camera formats and deliver encoded frames. It restricted the advertised video codec to H.264 and shared the camera source among connected viewers. This version depended on loading compatible native FFmpeg libraries into the device process.

A subsequent revision replaced those in-process bindings with an external FFmpeg executable. The .NET application now starts and supervises that process, receives its encoded output over a local RTP socket, and forwards reconstructed video samples to SIPSorcery. This changes the integration boundary: FFmpeg is invoked through a configured command line instead of a native-library binding. It also introduces an explicit RTP depacketization step inside the device service. The change does not remove the computational cost of video conversion.

In the final Linux configuration, FFmpeg captures MJPEG frames from the USB camera through Video4Linux2. These frames are decoded and then encoded as H.264 using libx264. The configured capture size is 1280 by 720 pixels at 30 frames per second. The superfast preset, zerolatency tuning, disabled B-frames, and controlled keyframe interval express the preference for timely delivery over maximum compression efficiency. This is a software encoding

pipeline; it does not pass a camera-generated H.264 stream directly to the browser.

FFmpeg sends the encoded video as RTP to a socket bound on the device's IPv6 loopback interface. The device service parses the packets and reconstructs the H.264 units that make up each frame. It assembles Annex-B samples and retains sequence and picture parameter sets so that they can accompany keyframes when required. SIPSorcery receives these encoded samples and handles the WebRTC transport to each browser peer. FFmpeg therefore captures and encodes the image, while SIPSorcery owns the negotiated WebRTC connection and its protected media transport.

Session establishment retains the signaling model developed by the browser prototype. The device and viewer join through SignalR using the same device identifier. The device creates a send-only video offer, the browser returns an answer, and both endpoints exchange ICE candidates. Candidates received before the remote description is available are held until they can be applied. Once the peer is connected, its video consumer is attached to the camera source. Capture starts for the first consumer and stops when the last consumer leaves.

Resource management forms part of the implementation. The camera source is shared so that additional viewers do not require independent camera captures and encoders. If FFmpeg exits while consumers remain, the service attempts to restart the process while retaining the local receive socket. Peer cleanup detaches the corresponding video consumer and closes its connection. These mechanisms support repeated test sessions and make process failures visible through diagnostic logging.

WHIP and WHEP were also investigated, but the working signaling path remains SignalR. The corresponding HTTP endpoints return Not Implemented, so the prototype does not provide a complete implementation of either protocol. Negotiated data channels provide a structure for control, commands, telemetry, and events, but complete vehicle actuation remains future work. Client and Server provide the code-level descriptions of these components and their responsibilities.

The implemented result is a low-latency video foundation for the RC vehicle. The documented setup produced an approximately 340 ms glass-to-glass observation using the Huawei 4G connection. This supports the feasibility of the target in that configuration; it is neither a statistical latency guarantee nor a 5G measurement. Completing vehicle control and evaluating genuine 5G connectivity remain necessary subsequent steps.

Client:

The client is the long-running application executed by the Raspberry Pi 5 installed in the vehicle. Its responsibility is to acquire and encode the camera stream, maintain presence with the application server, establish one WebRTC peer session per operator, publish video, and consume control data. The implementation separates the coordination path from the latency-sensitive path: SignalR carries device registration and WebRTC negotiation, while encoded video and vehicle-control messages use the selected WebRTC route directly. This separation prevents the application server from adding processing or queueing delay after a peer connection has been established.

Runtime Initialization and Configuration

The executable starts in `Program.Main` with the .NET Generic Host. `Startup.ConfigureServices` binds the `Device` configuration section to `DeviceOptions` and registers a deliberately small dependency graph: one SignalR `HubConnection`, one `CameraVideoSource`, one `WebRtcSessionManager`, one `WebRtcSignalingClient`, and the `WebRtcSignalingBackgroundService`. The services that own network, camera, and peer state are singletons so their lifetimes match the process and so the camera is not opened more than once for concurrent viewers. The hosted service provides a single orchestration loop instead of creating independent retry or heartbeat workers for each session.

`DeviceOptions` centralizes the stable device identifier, platform-specific camera path, capture resolution, frame rate, FFmpeg executable, bitrate, key-frame interval, RTP port, and argument templates. The connection string identifies the application server hub. `HubConnectionBuilder.WithAutomaticReconnect` keeps the signaling transport persistent, while `CameraOptions.GetPath` and `FfmpegOptions.GetArguments` select the Windows or Linux values without branching in the media loop. Configuration is validated by `CameraVideoSource.EnsureInitialised` before an offer advertises a video track; a missing device, missing argument template, or unknown placeholder therefore fails at session setup rather than after streaming starts.

Registration and Connectivity Lifecycle

`WebRtcSignalingBackgroundService.ExecuteAsync` connects the device to the SignalR hub and calls `WebRtcSignalingClient.JoinAsDeviceAsync`. The server adds the current SignalR connection to the group named by the device identifier, notifies any waiting operator, and returns the STUN and TURN configuration used by new peer connections. The worker stores that configuration in `WebRtcSessionManager.SetIceServers`, allowing connectivity policy to remain server-controlled while peer connections are created locally on demand.

A one-second `PeriodicTimer` drives the heartbeat without blocking the rest of the process. While disconnected, the same loop retries the hub connection every five ticks; while connected, it invokes `DeviceHeartbeat`. A successful automatic reconnect produces a new SignalR connection identifier and loses the previous group membership, so the `Reconnected` handler immediately rejoins the device group and refreshes the ICE server list. Existing WebRTC transports are not restarted solely because signaling reconnects: once negotiated, their media and data continue peer to peer unless the ICE transport itself fails.

Table 1. Raspberry Pi startup, registration, and signaling flow

Stage	Code reference	Action	Result or recovery
1	`Program.Main`	Build and run the Generic Host.	One process owns signaling, camera, and peer state.
2	`Startup.ConfigureServices`	Bind options and create singleton services.	Stable service lifetimes and one shared camera source.
3	`ConnectAsync`	Open the SignalR transport with automatic reconnect enabled.	Coordination channel becomes available.
4	`JoinAsDeviceAsync`	Join the group identified by `DeviceId`.	Server announces presence and returns ICE servers.
5	`SetIceServers`	Map supported STUN/TURN URLs into SIPSorcery configuration.	Future peers use centrally supplied connectivity policy.
6	`SendDeviceHeartbeatAsync`	Publish liveness once per timer tick.	Operators and server can observe availability.
7	`Reconnected` handler	Rejoin after SignalR assigns a new connection identifier.	Group membership and ICE configuration are restored.
8	Shutdown `finally`	Close peers and stop the hub connection.	Camera, sockets, and peer resources are released.

WebRTC Session Negotiation

When the hub reports `ClientJoined`, the background service asks `WebRtcSessionManager.CreateOfferAsync` to create a session for that browser connection. Any obsolete session under the same key is closed first. The manager validates the camera, creates an `RTCPeerConnection`, declares the four negotiated data channels, adds a send-only H.264 track, subscribes to ICE and connection-state events, and stores the resulting `WebRtcPeerSession` in a `ConcurrentDictionary`. It then creates the SDP offer, installs it as the local description, and returns it to `WebRtcSignalingClient.SendOfferAsync` for forwarding through the hub.

The answer path uses the browser connection identifier as the session key. `ApplyAnswer` installs the remote description inside a short per-session critical section, marks the session ready for remote candidates, copies the pending candidate list, and flushes it after leaving the lock. `AddIceCandidate` applies candidates immediately once the remote description exists; candidates that

arrive earlier are appended to the same session-local list. This ordering satisfies the WebRTC state machine without delaying unrelated peers. When the connection becomes connected, `AttachVideoAsync` registers its `SendVideo` delegate with the shared camera source. Failed, closed, or disconnected peers are removed through `ClosePeer` and detached from capture.

Latency-Oriented Code Complexity

The client is designed to minimize software-added latency rather than merely relying on WebRTC transport. The steady-state hot path contains one asynchronous UDP receive, RTP parsing and H.264 depacketization, one Annex-B access-unit assembly, and immediate dispatch to the active `SendVideo` delegates. It does not add an application-level frame queue between depacketization and transmission. Signaling, configuration expansion, candidate processing, and peer lookup occur outside this per-frame path.

One receive task owns `H264Depacketiser` and `CaptureStreamState`, so packet-order state requires no shared lock. The hot loop reads the multicast consumer delegate with `Volatile.Read` and never waits for the lock that protects capture startup, shutdown, and consumer changes. Locks around a peer protect only remote-description state and attachment bookkeeping. Network operations use `async` APIs, preventing a blocked hub, socket, or process shutdown from occupying the packet-processing thread.

Table 2. Latency-critical media path and computational complexity

Stage	Operation	Time complexity	Allocations or copies	Latency rationale	Code reference
Peer lookup	Find or remove session by connection ID.	Average O(1); worst case depends on hash contention.	O(1) session references.	Avoids linear scans as operators are added.	`ConcurrentDictionary.TryGetValue / TryRemove`
ICE setup	Flush k candidates after the answer is installed.	O(k).	O(k) temporary references.	Paid during negotiation, not for each video frame.	`ApplyAnswer`
RTP receive	Read, parse, validate payload type, and depacketize p payload bytes.	O(p) per packet.	Receive buffer and depacketizer state.	Single owner preserves order without per-packet locking.	`ReceiveAsync`
Frame assembly	Inspect NAL units and build an f-byte Annex-B access unit.	O(f) time and O(f) temporary memory.	One output array plus required byte copies.	No decode/re-encode stage; one bounded frame assembly.	`BuildAccessUnit`
Timestamp	Calculate duration from adjacent RTP timestamps.	O(1).	No frame-sized allocation.	Constant work per completed access unit.	`ComputeDurationRtpUnits`

Stage	Operation	Time complexity	Allocations or copies	Latency rationale	Code reference
Distribution	Invoke the encoded-sample delegate for n viewers.	O(n) per frame.	O(n) peer state; one shared encoded frame.	One encoder is shared instead of encoding separately per viewer.	`Volatile.Read`; `EncodedSampleDelegate.Invoke`
Attach/detach	Add or remove a viewer and start/stop capture at the boundaries.	O(n) multicast-delegate update; outside steady-state frame work.	New delegate invocation list when membership changes.	Viewer changes do not lock the per-frame dispatch path.	`AddConsumerAsync`; `RemoveConsumerAsync`

These bounds describe the additional computation performed by the client; they are not end-to-end latency measurements. Camera exposure, USB transfer, H.264 encoding, radio scheduling on the mobile network, the selected ICE route, congestion control, browser jitter buffering, decoding, display refresh, and physical actuation remain independent contributors. The code reduces the delay under its control by keeping setup work off the media path, bounding the work per packet and frame, avoiding repeated encoding, and preventing stale control messages from accumulating.

Camera-to-WebRTC Video Pipeline

Capture starts only when the first connected peer is attached. `CameraVideoSource.StartCapture` binds an IPv6 loopback UDP socket before spawning FFmpeg, obtains the actual port when zero requests an ephemeral port, and expands the configured command line with that port and RTP payload type. FFmpeg reads MJPEG from Video4Linux2 or DirectShow, encodes H.264 constrained baseline video, and writes RTP packets to the local socket. Binding first prevents an initial burst from being lost between process start and socket creation.

The encoder configuration makes latency an explicit trade-off. `-fflags nobuffer` and `-flags low_delay` discourage demuxer and codec buffering; `-preset superfast` trades compression efficiency for encoding speed; and `-tune zerolatency` removes look-ahead behavior intended for offline compression. `-bf 0` disables B-frames that would require frame reordering. A controlled GOP provides regular IDR recovery points, while a rate-control buffer derived as one quarter of the configured bitrate limits bursts that would otherwise wait in the mobile modem queue. RTP packets are limited to 1200 bytes to reduce IP fragmentation risk across Internet paths.

`ReceiveAsync` parses the loopback RTP stream and passes payloads to `H264Depacketiser`. At the marker packet, the depacketizer returns the NAL units belonging to the completed frame. `BuildAccessUnit` caches SPS and PPS parameter sets and prepends them to an IDR frame when the encoder did not

repeat them, then performs a single sized allocation for the Annex-B access unit. SIPSorcery's `SendVideo` repacketizes that encoded unit using the payload type negotiated for each peer. Because the Raspberry Pi never decodes the camera output before WebRTC transmission, it avoids an entire decode, raw-frame copy, and second encode cycle.

Control, Commands, Telemetry, and Events

The client and browser create the same negotiated SCTP streams with fixed identifiers, so channel creation does not depend on an in-band `datachannel` event. Each stream is assigned according to whether freshness or guaranteed delivery is more important. Control state and telemetry use unordered delivery with zero retransmissions: if a packet is lost or delayed, the next state supersedes it instead of waiting behind stale data. Commands and events retain reliable, ordered delivery for operations whose sequence must be preserved. Payload schemas remain an application-level contract and are not imposed by the transport layer.

Table 3. Negotiated WebRTC data-channel contract

Channel	ID	Primary direction	Delivery policy	Target responsibility	Latency behavior
`control`	0	Browser to vehicle	Unordered; `maxRetransmits = 0`	Continuous steering and throttle state.	New state is not blocked by an obsolete lost packet.
`commands`	1	Browser to vehicle	Reliable and ordered	Discrete configuration or actions that must arrive in sequence.	Accepts retransmission delay to preserve correctness.
`telemetry`	2	Vehicle to browser	Unordered; `maxRetransmits = 0`	Frequently refreshed measurements and operating state.	A newer sample supersedes a delayed older sample.
`events`	3	Bidirectional	Reliable and ordered	Lifecycle and exceptional events requiring consistent order.	Delivery correctness is prioritized over minimum delay.

In the target control flow, the browser samples the gamepad, normalizes steering and throttle values, serializes the latest state, and transmits it through `control`. The Raspberry Pi validates the message and updates the motor and steering interface without routing it through ASP.NET Core. The direct data-channel path minimizes network hops, while the zero-retransmission policy ensures that a delayed command cannot be applied after a newer operator input.

Runtime Resilience

A single `CameraVideoSource` supports all peers through a multicast encoded-sample delegate. The first consumer starts capture and the last consumer stops it, preventing an idle FFmpeg process from retaining the camera or filling the

socket. If FFmpeg exits while consumers remain, `HandleFfmpegExitedAsync` keeps the bound UDP socket and receive task alive, waits one second, and replaces only the failed process. Reference-identity checks reject a restart callback that belongs to a deliberately stopped or superseded process.

Shutdown swaps the active capture reference, terminates and drains FFmpeg, cancels the receive loop, and only then disposes the socket and cancellation source. The last fifty FFmpeg warning lines are retained for failure diagnosis without unbounded memory growth. Peer cleanup removes the session atomically, detaches its video delegate, closes the `RTCPeerConnection`, and leaves other viewers and the shared encoder unaffected.

Server:

The server side consists of two cooperating applications delivered as one web system. ASP.NET Core provides the coordination boundary: HTTP APIs, device discovery, SignalR signaling, ICE configuration, health checks, OpenAPI, and static application hosting. React provides the operator boundary in a standard browser: device selection, session state, WebRTC negotiation, video presentation, connection diagnostics, and control input. The backend introduces the peers but remains outside the video and control paths after negotiation.

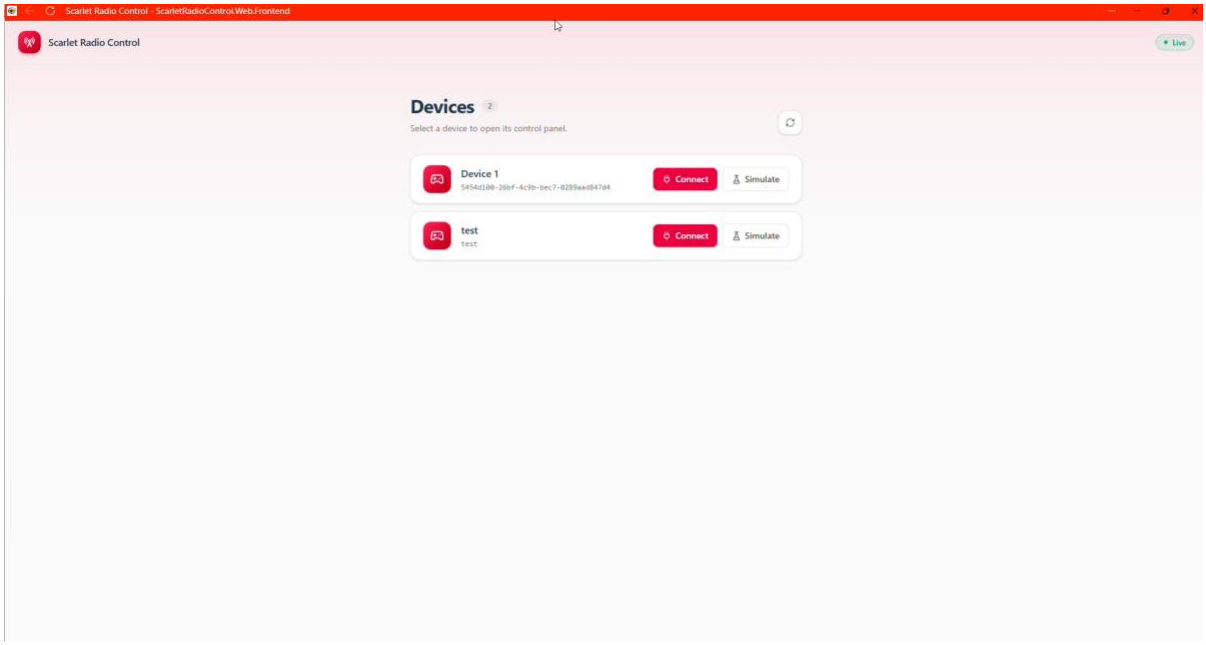

Figure 9 Device Selection Interface

Application Composition

`Program.Main` creates the ASP.NET Core host and delegates configuration to `Startup`. The middleware pipeline redirects HTTP to HTTPS, converts unhandled failures to problem responses, performs routing and authorization, and then maps controllers, `/health`, `/hubs/web-rtc-hub`, the OpenAPI document, static assets, and the SPA fallback. `MapFallbackToFile('/index.html')` lets React Router own browser routes such as `/device/:deviceId/control` without requiring a matching server controller.

The React entry point mounts `App` inside `BrowserRouter` and `SignalRProvider`. `App` declares the device list, live control, simulation, and not-found routes. During development, `vite.config.ts` proxies `/api` and `/hubs` to the HTTPS ASP.NET Core process, including WebSocket upgrades. In a published build, ASP.NET Core serves the same compiled assets. The PWA configuration caches the

application shell but excludes API and hub navigation from fallback handling so real-time requests are never replaced by cached HTML.

Device Discovery and Operator Entry

The discovery contract is `GET /api/v1/devices`, which returns `Device` records containing the identifier used for signaling groups and the human-readable name displayed to the operator. The OpenAPI description is the source for the generated Kiota TypeScript client. `useApiClient` creates one `FetchRequestAdapter` with the application authentication provider and exposes the typed request builder to React components, preventing HTTP paths and JSON shapes from being duplicated manually in the UI.

`Index` requests the device catalog on entry and every ten seconds, sorts records by name or identifier, and represents loading, live, reconnecting, offline, empty, and manual-refresh states explicitly. Selecting Connect navigates to the live `Control` route with the device identifier in the URL; selecting Simulate opens `ControlTest`. This keeps discovery traffic on ordinary HTTP and creates a SignalR/WebRTC session only after the operator selects a vehicle.

Signaling Session Coordination

`SignalRProvider` creates one application-wide `HubConnection` to `/hubs/web-rtc-hub`, enables automatic reconnection, and exposes both the connection object and a simple connected state through React context. `Control` invokes `JoinAsClient(deviceId, null)` once the hub and browser `RTCPeerConnection` are ready. `WebRtcHub.JoinAsClient` adds that connection to the device group, notifies the Raspberry Pi through `ClientJoined`, and returns the same ICE server collection used by the device.

The strongly typed `IWebRtcClient` contract defines the events that the hub can emit: client or device joins, heartbeats, offers, answers, and ICE candidates. `JoinAsDevice` performs the symmetric registration for the Raspberry Pi. The device identifier scopes discovery and group notifications, while the SignalR connection identifier targets a particular browser peer. This distinction allows one vehicle to announce itself to a group while SDP and candidate messages are forwarded only to the intended session.

Offer, Answer, and ICE Exchange

The Raspberry Pi is the offerer. After `ClientJoined`, it creates an offer and invokes `SendOffer` with the device identifier, the target browser connection identifier, and the SDP description. The hub forwards the description through `ReceiveOffer` and includes the sender connection identifier. `Control` installs the offer as its remote description, creates and installs an answer, and returns it through

`SendAnswer`. The device then applies the answer to the peer session associated with that browser.

Both endpoints gather candidates asynchronously. The hub's `SendIceCandidate` method forwards each candidate to the supplied target connection and does not inspect or transform it. The React client queues candidates that arrive before `remoteDescription` and flushes them immediately after the offer is installed; the Raspberry Pi uses the same ordering rule around the answer. Once ICE selects a route and DTLS completes, the hub retains session coordination duties but receives neither camera frames nor control messages.

Table 4. End-to-end Raspberry Pi, ASP.NET Core, and React negotiation flow

Step	Raspberry Pi client	ASP.NET Core	React browser	Result
1	Connect and invoke `JoinAsDevice`.	Add connection to the device group and return ICE servers.	Poll the device catalog.	Vehicle is discoverable.
2	Send periodic heartbeat.	Forward liveness to other group members.	Select the device and open `Control`.	Operator chooses when to create a real-time session.
3	Wait for `ClientJoined`.	Process `JoinAsClient`; notify the device; return ICE servers.	Join as client and configure `RTCPeerConnection`.	Both endpoints use a consistent ICE policy.
4	Create and install the SDP offer.	Forward `SendOffer` to the target connection.	Receive and install the offer.	Browser learns the proposed media and data channels.
5	Wait for the answer.	Forward `SendAnswer` to the device connection.	Create, install, and send the SDP answer.	Compatible session parameters are selected.
6	Gather and send ICE candidates.	Forward candidates without entering the transport path.	Gather, send, queue, and apply candidates.	ICE tests direct and relay routes.
7	Attach `SendVideo` after `connected`.	Remain outside media and control traffic.	Handle `ontrack` and play the stream.	Video flows over the selected WebRTC route.
8	Consume control and publish telemetry/events.	Maintain only coordination state.	Sample input and use negotiated data channels.	Bidirectional real-time operation is peer to peer.
9	Close the peer and detach capture on failure or termination.	Observe disconnects and group changes.	Release the browser peer and UI state.	Session resources return to a clean state.

Browser Media and Control Session

`useRtcPeerConnection` creates one stable browser peer for the lifetime of the control component. It declares the same four negotiated channels and identifiers as `WebRtcSessionManager`, eliminating channel-order races. `Control` registers SignalR handlers for the device offer and ICE candidates, assigns `onicecandidate` for browser candidates, and assigns `ontrack` to place the first received `MediaStream` into the video element. `autoplay`, `muted`, and `playsInline` allow the live view to start without a separate desktop player.

When `connectionState` becomes connected, the browser reads WebRTC statistics, follows the selected transport's candidate-pair identifiers, and records the local and remote candidate types. These values expose whether the session is direct or relayed without changing the media path. `useGamepad` observes browser connection and disconnection events and supplies the selected controller. The target control loop sends only current normalized state on the low-latency `control` channel and reserves the reliable streams for discrete commands and events.

Validation and Standardized Interfaces

`ControlTest` exercises the signaling flow without the physical vehicle. It joins the same hub as a device, publishes heartbeats, captures a looping countdown video with `captureStream`, adds the resulting track to a browser `RTCPeerConnection`, creates the offer after `ClientJoined`, and handles answers and ICE candidates. Because it follows the same hub method names and candidate-order rules, it validates browser, backend, NAT traversal, and media presentation independently of Raspberry Pi camera or motor hardware.

The device API also exposes WHIP and WHEP entry points using `application/sdp`. WHIP standardizes publisher ingestion and WHEP standardizes viewer egress through HTTP-created session resources, complementing the interactive SignalR flow. Their controller actions validate the SDP offer, establish the corresponding server-side peer role, return the SDP answer with `201 Created` and a session location, accept ICE updates, and delete the resource at termination. They are appropriate for interoperable one-way integrations; the primary vehicle-control path retains SignalR because it coordinates a bidirectional video and data-channel session.

Conclusions:

The principal objective of this work was to deliver a remotely viewable video stream with an end-to-end latency below 400 ms. Under the prototype test conditions, the observed glass-to-glass video delay remained below that threshold. This result demonstrates that WebRTC is a technically viable real-time media foundation for an Internet-connected RC vehicle, while remaining specific to the equipment, network route, and configuration used during the tests.

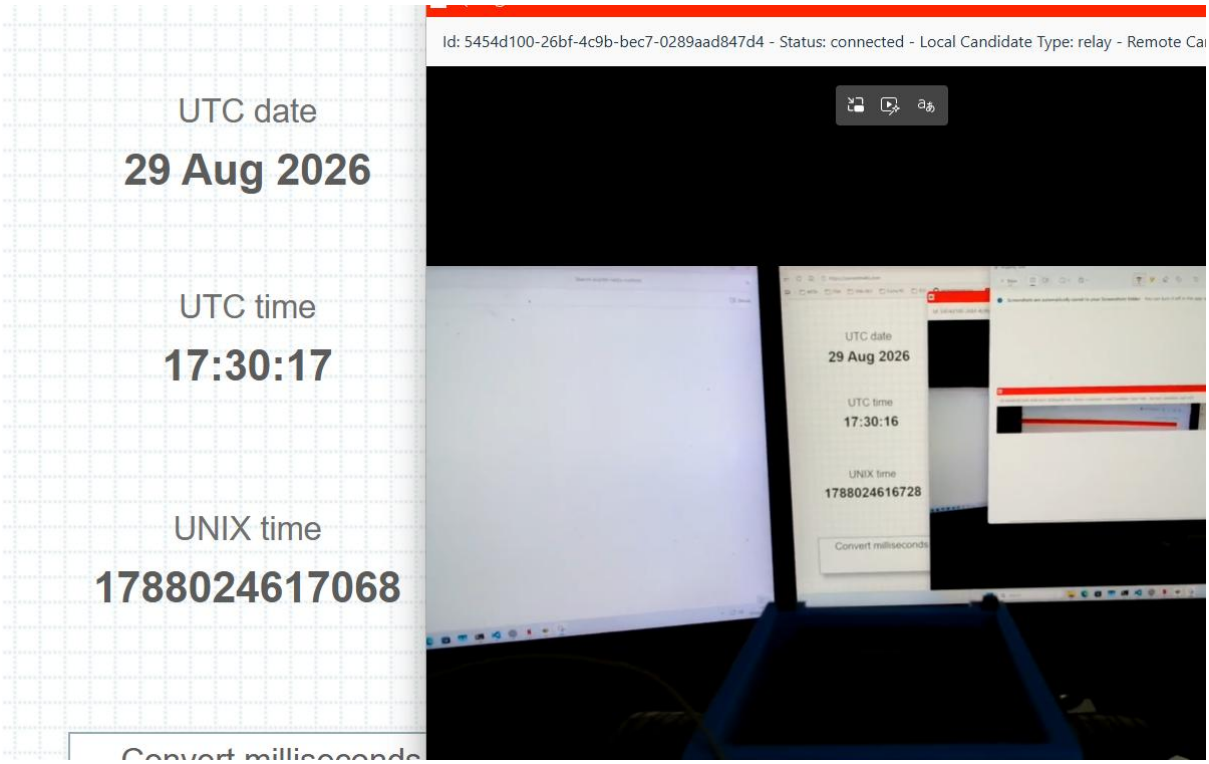

Figure 10 Glass-To-Glass latency test (340ms)

The result is consistent with the architecture selected for the prototype. ASP.NET Core and SignalR provide registration, session coordination, and signaling, but remain outside the media path after ICE selects a route. Video can therefore travel directly between the vehicle and the browser, while the Raspberry Pi pipeline avoids an additional application-level frame queue and uses low-latency H.264 settings. Native IPv6 can increase the probability of selecting a direct peer-to-peer candidate because it may avoid IPv4 NAT and carrier-grade NAT conditions that force a TURN relay. IPv6 is not inherently faster; the latency benefit arises when ICE avoids the additional relay hop.

Further latency reductions remain possible. The current Raspberry Pi 5 pipeline captures MJPEG and encodes it in software with libx264. Hardware H.264 output is expected to reduce encoding delay, CPU load, and memory traffic by removing that conversion stage, provided the encoder does not introduce

unnecessary buffering. Likewise, replacing the prototype's 4G connection with a genuine 5G deployment, preferably 5G Standalone under suitable coverage and routing conditions, could reduce radio-access latency and jitter. This is an expected improvement that must be measured end to end rather than assumed from the network label alone.

The achieved objective is limited to the video and connection foundation. Complete remote driving has not yet been achieved. The software contains the signaling, media path, gamepad-detection hook, and negotiated data-channel scaffolding, but it does not yet implement the complete browser control loop, device-side command validation, or physical steering and motor actuation. The prototype therefore validates low-latency remote observation and the communication architecture, while bidirectional vehicle operation remains the principal next stage.

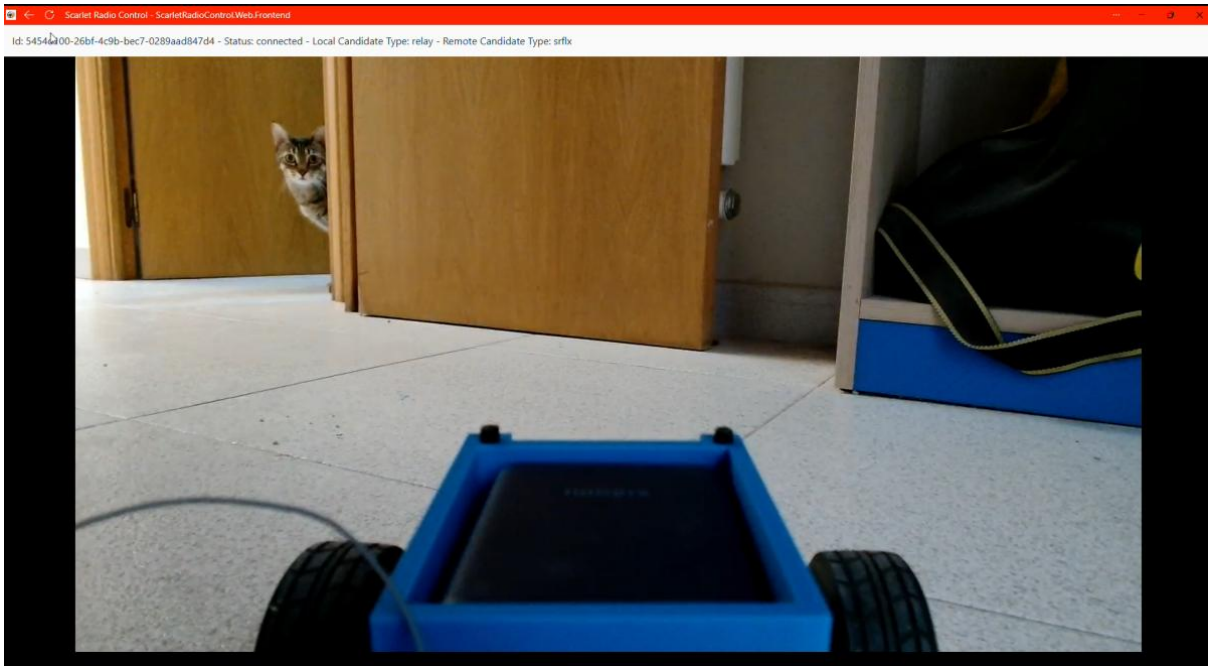

Figure 11 Live vehicle video-stream

Future Work:

Complete Remote Control and Safety

The first priority is to complete the control path already anticipated by the negotiated WebRTC data channels. The browser should sample the selected gamepad at a fixed cadence and continuously transmit a versioned control message containing normalized steering and throttle values, a sequence number, and a timestamp. The vehicle must reject malformed, out-of-range, or stale state before converting the latest valid message into PWM or other actuator signals for steering and propulsion.

Safety behavior must be part of the initial implementation. Loss of the data channel, expiry of a command timeout, an invalid message, or an explicit emergency-stop command must immediately return steering and throttle to a defined neutral state. The complete path from gamepad input to physical actuation should then be tested end to end for responsiveness, failure recovery, and predictable shutdown.

IPv6 and Mobile-Network Validation

The device, browser environment, signaling service, and TURN service should be deployed in a dual-stack configuration. Each session should record the address family, candidate types, selected ICE pair, and whether the connection is direct or relayed. Comparisons between IPv4 direct, IPv6 direct, and TURN-relayed sessions should measure connection success, latency, jitter, and packet loss to determine whether IPv6 materially improves the available route.

The same campaign should replace the current 4G access with a real 5G connection and distinguish 5G Non-Standalone and Standalone modes where possible. Testing must cover direct and relayed operation, mobility, uplink consistency, latency distribution, jitter, packet loss, and video stability rather than relying on nominal capabilities.

Hardware H.264 Camera

A compact camera that exposes an H.264 stream directly should be evaluated to remove the Raspberry Pi's MJPEG-to-libx264 stage. The Arducam B0200 specification lists H.264 output at 1920 x 1080 and 30 frames per second. Validation must confirm profile compatibility, SPS/PPS handling, keyframe interval, bitrate control, FFmpeg/V4L2 support, CPU usage, and measured glass-to-glass latency. Hardware encoding should be adopted only if the

camera and capture path do not add buffering that offsets the expected improvement.

Protected Electrical Design

The temporary wiring should be replaced with a documented power-distribution design. The Raspberry Pi and communication electronics should receive power through an appropriately rated DC-DC regulator and a dedicated fused branch, with reverse-polarity protection, transient suppression, bulk and ceramic decoupling, secure connectors, and a grounding arrangement that limits motor noise. Bench tests should reproduce startup and motor-stall current peaks while monitoring regulated voltage, brownout indications, component temperature, and recovery after a fault.

Mechanical Integration

The 3D-printed supports should be redesigned as modular mounts for the Raspberry Pi, camera, mobile modem, power regulator, and control electronics. The next design should provide positive fastening, vibration resistance, cable strain relief, protected routing, cooling airflow, maintenance access, and balanced weight distribution. Separate replaceable mounts will allow individual components to change without redesigning the complete chassis assembly.

ESP32-P4 Integrated Rust Platform

A longer-term path is a custom controller board based on a production ESP32-P4 revision. Its MIPI-CSI camera interface, image signal processor, 1080p30 hardware H.264 encoder, GPIO, and motor-control PWM could combine camera capture, video encoding, safety logic, and vehicle actuation in a smaller, lower-power platform.

The ESP32-P4 firmware will be developed in Rust. An ESP-IDF-backed architecture will retain access to vendor multimedia and networking components while application logic, control parsing, concurrency, and fail-safe state machines remain in safe Rust. The esp-idf-sys bindings and esp-idf-hal should be used for the riscv32imafc-esp-espidf target. H.264, MIPI-CSI, USB, Ethernet, and modem APIs without mature native Rust abstractions should be isolated behind small reviewed unsafe FFI modules.

ESP32-P4 has no integrated Wi-Fi or Bluetooth. The custom board must therefore connect to the 5G modem through USB or Ethernet, or add an ESP32-C5/C6 communication processor. Development should proceed incrementally through actuator fail-safe behavior, camera capture, hardware H.264 output, modem networking, ICE/DTLS/WebRTC feasibility, memory usage, thermal behavior, and final end-to-end latency.

Final System Evaluation

The completed prototype should be evaluated with a matrix covering software and hardware encoding, 4G and available 5G modes, IPv4 and IPv6, and direct and TURN-relayed routes. Reported results should include the distribution of glass-to-glass delay, control-response time, jitter, loss, bitrate, CPU load, temperature, electrical stability, and fail-safe activation. This will show which improvements provide measurable benefits and whether the final vehicle satisfies both observation and control requirements.

References:

[1] H. T. Alvestrand, "Overview: Real-Time Protocols for Browser-Based Applications," IETF, RFC 8825, Jan. 2021.

[2] J. Uberti, C. Jennings, and E. Rescorla, "JavaScript Session Establishment Protocol (JSEP)," IETF, RFC 9429, Apr. 2024.

[3] A. Keranen, C. Holmberg, and J. Rosenberg, "Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator Traversal," IETF, RFC 8445, Jul. 2018.

[4] M. Petit-Huguenin et al., "Session Traversal Utilities for NAT (STUN)," IETF, RFC 8489, Feb. 2020.

[5] T. Reddy, A. Johnston, P. Matthews, and J. Rosenberg, "Traversal Using Relays around NAT (TURN)," IETF, RFC 8656, Feb. 2020.

[6] R. Jesup, S. Loreto, and M. Tuexen, "WebRTC Data Channels," IETF, RFC 8831, Jan. 2021.

[7] Y.-K. Wang, R. Even, T. Kristensen, and R. Jesup, "RTP Payload Format for H.264 Video," IETF, RFC 6184, May 2011.

[8] A. B. Roach, "WebRTC Video Processing and Codec Requirements," IETF, RFC 7742, Mar. 2016.

[9] S. Garcia Murillo and A. Gouaillard, "WebRTC-HTTP Ingestion Protocol (WHIP)," IETF, RFC 9725, Mar. 2025.

[10] S. Garcia Murillo, C. Chen, and D. Jenkins, "WebRTC-HTTP Egress Protocol (WHEP)," IETF Internet-Draft, draft-ietf-wish-whep-04, Jun. 2026, work in progress.

[11] World Wide Web Consortium (W3C), "WebRTC: Real-Time Communication in Browsers," W3C Recommendation, Mar. 13, 2025.

[12] MDN Web Docs, "WebRTC API," Mozilla. Accessed: Aug. 29, 2026.

[13] WebRTC Project, "Getting Started with Peer Connections," Google. Accessed: Aug. 29, 2026.

[14] FFmpeg Project, "FFmpeg Devices Documentation," sec. 3.18, Video4Linux2 input device. Accessed: Aug. 29, 2026.

[15] FFmpeg Project, "FFmpeg Codecs Documentation," sec. 9.19, libx264 encoder. Accessed: Aug. 29, 2026.

[16] Raspberry Pi Ltd, "Raspberry Pi 5 Product Brief," Apr. 2026.

[17] Linux Kernel Project, "Video for Linux API, Version 2 - V4L2 API Specification," rev. 4.5. Accessed: Aug. 29, 2026.

[18] Microsoft, "Use Hubs in ASP.NET Core SignalR," Microsoft Learn. Accessed: Aug. 29, 2026.

[19] ETSI and 3GPP, "System Architecture for the 5G System (5GS)," ETSI TS 123 501 V19.8.0, 3GPP TS 23.501 Release 19, Jul. 2026.

[20] International Telecommunication Union Radiocommunication Sector (ITU-R), "Minimum Requirements Related to Technical Performance for IMT-2020 Radio Interface(s)," Report ITU-R M.2410-0, 2017.